

Faculty of Electrical Engineering - University of Sarajevo

Mentor Prof. dr. Izudin Džafić

Student Almir Mustafić

GaussNewtonFit

**Gauss-Newton Method for Nonlinear Parameter Estimation
and Model Fitting**

Executive Summary

GaussNewtonFit is a C++14 system for nonlinear least-squares parameter estimation. The system implements Gauss-Newton and full Newton-Raphson solvers. It compares both methods under the same models, datasets, noise levels, initial parameter values, and stopping criteria.

The system generates synthetic data for exponential decay, logistic growth, and sinusoidal models. Users can run a fit from the desktop interface and inspect the fitted curve, residual history, estimated parameters, and execution log. The command-line test harness also validates analytical derivatives and runs a 36-experiment comparison.

The project studies solver behavior under controlled conditions. The comparison measures convergence, failure, iteration count, residual norm, execution time, and parameter error.

1. About the Project

GaussNewtonFit solves nonlinear parameter estimation problems by minimizing least-squares residuals. The project uses two optimization methods for the same fitting tasks. This setup isolates differences between Gauss-Newton and full Newton.

- The system fits an exponential decay model with two parameters.
- The system fits a logistic growth model with three parameters.
- The system fits a sinusoidal model with three parameters.
- The system generates synthetic data with configurable Gaussian noise.
- The system records the residual norm during every solver iteration.
- The system exports experiment results and run reports to files.

2. System Architecture Overview

GaussNewtonFit separates data generation, model evaluation, solver logic, testing, visualization, and export functions.

- The data generator creates synthetic observations for the selected model.
- The model layer evaluates predictions and analytical derivatives.
- The solver layer runs Gauss-Newton or full Newton.
- The test harness checks derivatives and runs batch experiments.
- The GUI shows the model fit and residual history.
- The export layer writes logs, reports, CSV files, and plot states.

3. Technology Stack

- C++14 implements the models, solvers, tests, file export, and application logic.
- CMake manages the project build.
- The Matrix library performs matrix operations and solves linear systems.
- natGUI provides the desktop interface.
- natPlot renders the model-fit and convergence plots.
- The C++ standard library provides random number generation, timing, mathematical functions, and file operations.

4. Models and Datasets

The project uses synthetic datasets only. Each dataset contains 10 equidistant sample points on the interval from 0 to 5. The experiments use Gaussian noise levels of 0.00, 0.05, and 0.20 with a fixed random seed of 42.

- The exponential dataset uses $y = 2.0 \exp(-0.3 t) + e$. The ground-truth parameters are 2.0 and 0.3.
- The logistic dataset uses $y = 10.0 / (1 + \exp(-1.5(t - 2.5))) + e$. The ground-truth parameters are 10.0, 1.5, and 2.5.
- The sinusoidal dataset uses $y = 3.0 \sin(2.0 t + 0.5) + e$. The ground-truth parameters are 3.0, 2.0, and 0.5.

5. Optimization Methods

GaussNewtonFit implements two solvers for the same nonlinear least-squares objective.

5.1 Gauss-Newton

Gauss-Newton uses first-order derivatives. The method forms the matrix $J^T J$ and solves a linear system for the parameter update. The method does not use second-order residual derivatives.

The solver computes the residual vector and the Jacobian at the current parameter estimate. It solves the normal equations and updates the parameter vector. It repeats this process until a stopping condition is met.

5.2 Full Newton-Raphson

Full Newton uses first-order and second-order derivatives. The method forms the exact least-squares Hessian by combining $J^T J$ with the residual-weighted Hessians of the model residuals.

The solver computes the residual vector, Jacobian, and residual Hessians at the current parameter estimate. It solves the resulting linear system and updates the parameter vector. It repeats this process until a stopping condition is met.

5.3 Shared Stopping Criteria

Both solvers use the same stopping rules. A run succeeds when the maximum parameter step falls below the tolerance or when the residual norm falls below the tolerance. A run is marked as failed when the maximum iteration count is reached without meeting the required residual condition.

6. Analytical Derivatives and Validation

The system implements analytical Jacobians for all three models. Full Newton also uses analytical residual Hessians for every data point.

The test suite checks these derivatives against central finite differences. The derivative check uses a perturbation of $1e-6$. The accepted Jacobian error is below $1e-4$. The accepted Hessian error is below $3e-3$.

7. Experiment Design

The automated comparison runs 36 experiments. The grid combines three models, two solvers, three noise levels, and two initial parameter offsets.

- The project tests the exponential, logistic, and sinusoidal models.
- The project tests Gauss-Newton and full Newton.
- The project tests noise levels of 0.00, 0.05, and 0.20.
- The project tests initial parameter offsets of 0.05 and 1.0.

The system records the iteration count, final residual norm, estimated parameters, parameter error, execution time, and failure status for each run.

8. Key Results

Both solvers converge from close initial values across all three models. These runs usually finish within 3 to 7 iterations and reach similar parameter accuracy.

The exponential model separates the methods under distant initialization. Gauss-Newton converges in about 7 to 11 iterations. Full Newton reaches the 50-iteration limit and produces a large parameter error in the tested configuration.

The logistic model produces the opposite result under distant initialization. Gauss-Newton stalls in the flat saturation region and produces large parameter errors. Full Newton converges in about 7 to 8 iterations because the second-order term provides curvature information.

The sinusoidal model remains sensitive to the initial parameter values. Both methods can converge to incorrect phase or frequency regions when the initial values are not suitable.

Higher noise increases the final residual norm because the fitted model cannot remove the added measurement noise.

9. Inputs and Outputs

The application accepts a model selection, a solver selection, a noise level, and initial parameter values. The command-line tests also use a maximum iteration count, a tolerance, and a fixed random seed.

The application returns an estimated parameter vector and a residual history. It also reports the iteration count, final residual norm, execution time, and failure status.

The program writes several result files during testing and export.

- The files `exponential.csv`, `logistic.csv`, and `sinusoidal.csv` store generated datasets.
- The file `comparison_results.csv` stores the full experiment grid.
- The file `comparison_summary.csv` stores side-by-side solver comparisons.
- The file `fit_log.txt` stores the GUI execution history.
- The file `run_export.txt` stores a formatted report for one run.
- The files `fit_plot.xml` and `convergence_plot.xml` store plot states.

10. Target Users

- Students can use the project to study nonlinear least-squares methods.
- Instructors can use the project to demonstrate convergence and failure behavior.
- Researchers can use the project to compare solver behavior under controlled conditions.
- Engineers can use the project to test nonlinear model fitting.

11. Existing Alternatives

Several existing libraries and applications solve nonlinear least-squares problems.

- SciPy provides nonlinear least-squares and curve-fitting methods in Python.
- Ceres Solver provides nonlinear optimization tools in C++.
- GNU Scientific Library provides numerical fitting and optimization methods in C.
- MATLAB Optimization Toolbox provides nonlinear model-fitting functions.

12. Why This Project Matters

GaussNewtonFit focuses on the direct comparison of Gauss-Newton and full Newton. Both solvers use the same models, data, matrix operations, stopping rules, and result metrics.

The project exposes the mathematical difference between the two methods. Gauss-Newton uses $J^T J$. Full Newton adds the residual-weighted second-derivative term.

The project also shows failure cases that general-purpose libraries may hide through damping, trust regions, or line-search procedures. The experiments therefore show how the base methods behave before those additional mechanisms are added.

The code uses analytical derivatives for the tested models. The test harness checks those derivatives with finite differences before running the solver comparisons.

13. Graphical User Interface

The desktop interface lets the user select a model and an optimization method. The user sets the noise level and the initial parameter values before running the fit.

The left plot shows the generated data and the fitted model. The right plot shows the residual norm by iteration. The status line reports the iteration count and final residual norm. The execution log lists recent runs. The export control writes the current log to a file.

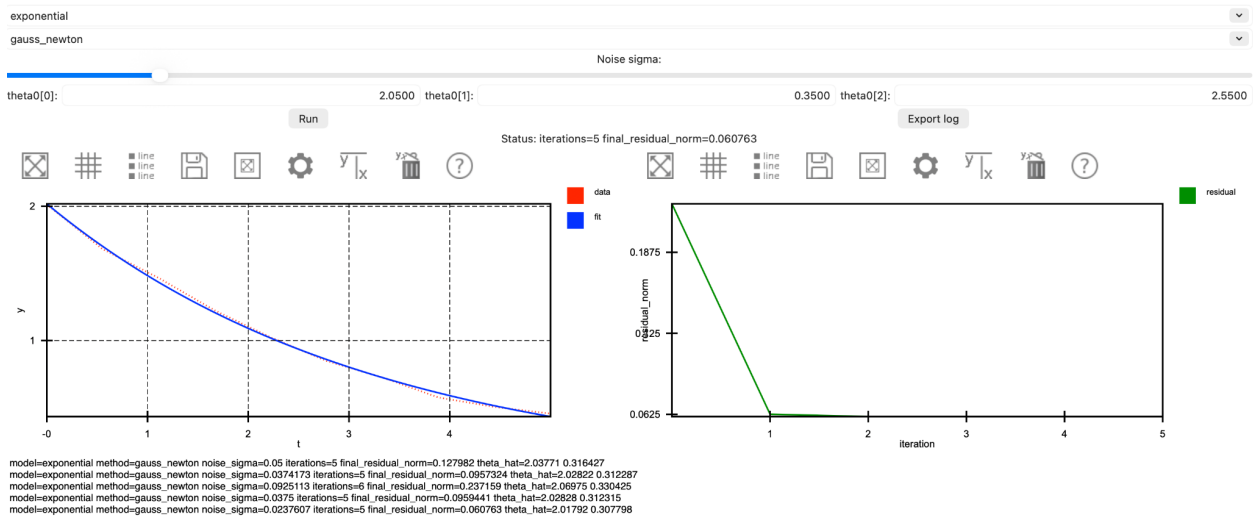


Figure 1 - GaussNewtonFit desktop interface

14. Current Limitations

- The system uses synthetic datasets and does not import measured datasets through the GUI.
- The GUI provides three parameter input fields and therefore limits interactive use to models with up to three parameters.
- The solvers use unweighted least squares even though the dataset structure stores noise information.
- The solvers use full update steps without damping or line search.
- The linear solver marks the run as failed when the system matrix becomes singular or poorly conditioned.

15. Future Work

- The project can add CSV or TSV import for measured datasets.
- The project can add weighted least squares.
- The project can add Levenberg-Marquardt damping or trust-region control.
- The project can add covariance estimates, parameter standard errors, confidence intervals, and R-squared values.
- The GUI can support models with more than three parameters.
- The application can add support for user-defined model equations.

16. Conclusion

GaussNewtonFit implements Gauss-Newton and full Newton for the same nonlinear fitting tasks. The project compares how both methods respond to noise and initial parameter values. The results show that neither method performs best in every tested case. The application combines the solver comparison with derivative checks, batch experiments, plots, and file export.